

Platform 2.0 — Authentication & tenant user lifecycle

Design document (not implemented) · SandBox repo · Generated from `platform-authentication-standard.md`

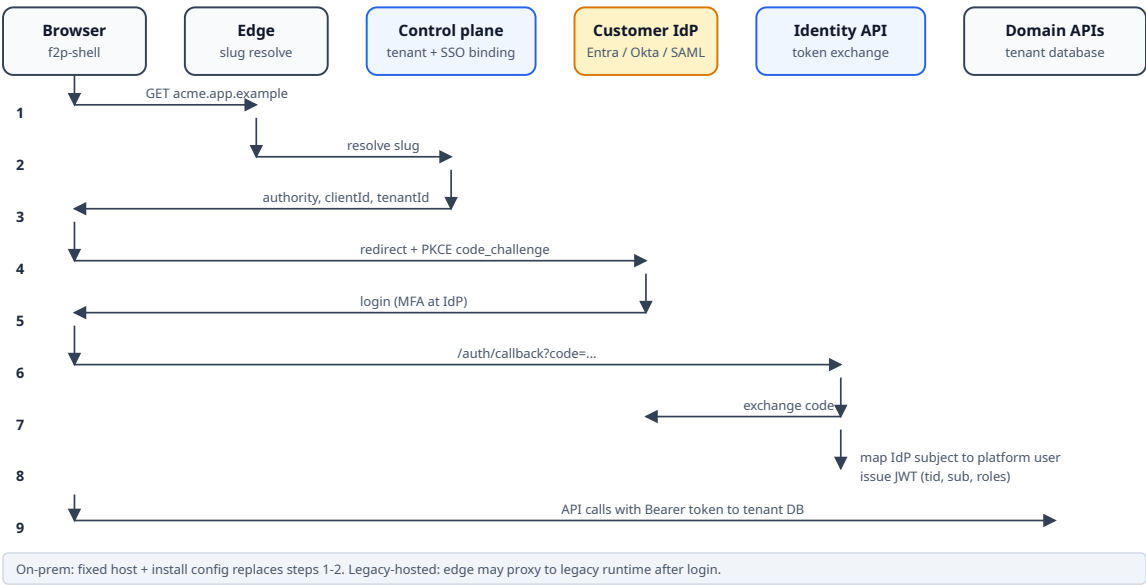
1. Summary

- **One product binary** for cloud SaaS and on-prem; behaviour driven by deployment profile and install config.
- **OIDC + PKCE** for browser login; platform issues its own JWT with `tid`, `sub`, and roles.
- **One database per tenant** — optional shared SQL Server *instance*, never a shared multi-tenant database.
- **Identity module** (`F2P.Identity` / `@f2p/identity`) owns login accounts, roles, and tenant-admin user management.
- **Admin backoffice** (separate) provisions tenants; **tenant admins** manage operational users who book hours and edit planning data.

2. Cloud authentication flow

Platform 2.0 - Cloud login flow (native runtime)

OIDC Authorization Code + PKCE | platform-issued JWT | one database per tenant



Step	Actor	Action
1	Browser	User opens <code>https://{slug}.app.example</code>
2	Edge	Resolve slug via control plane → tenantId + OIDC settings
3	Shell	Redirect to customer IdP with PKCE <code>code_challenge</code>
4	IdP	User authenticates (MFA enforced by customer IdP)
5	Identity API	Exchange auth code; map IdP subject → platform user
6	Identity API	Issue platform JWT (<code>tid</code> , <code>sub</code> , roles)
7	Domain APIs	Validate JWT; open connection to tenant's dedicated database

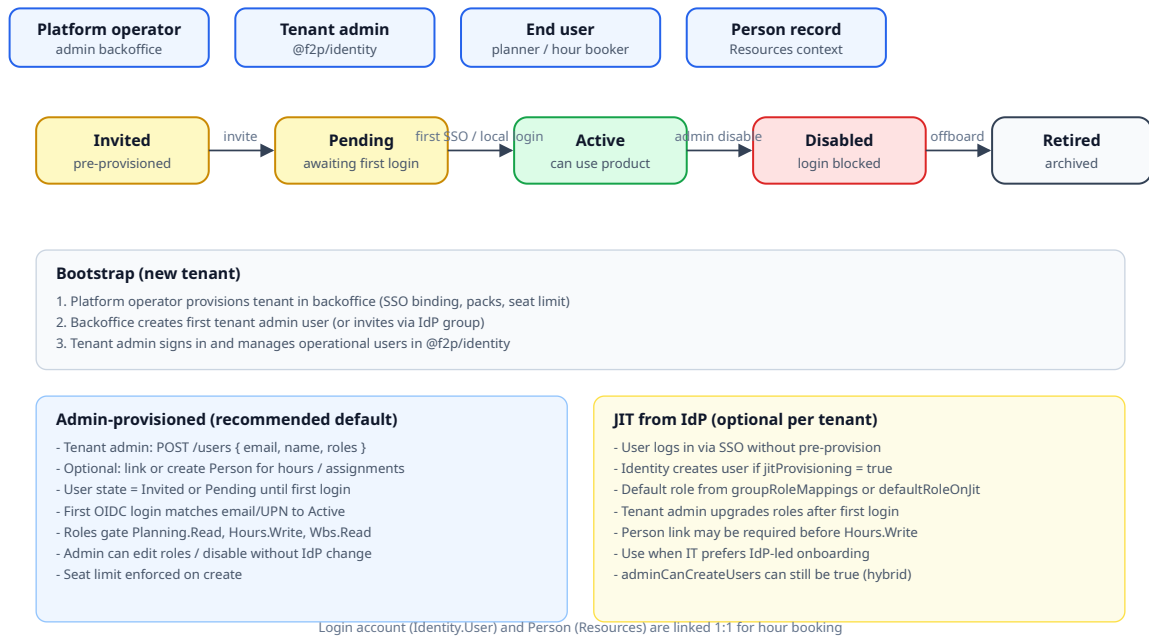
On-prem variant

Fixed host and `tenantId` from install config replace slug lookup. Same OIDC sequence; signing keys and user store may be customer-managed.

3. Tenant user lifecycle

Platform 2.0 - Tenant user lifecycle

Tenant administrators manage operational users via @f2p/identity



Three user layers

Layer	UI / API	Responsibility
Platform operator	Admin backoffice	Provision tenant, bind SSO, enable packs, bootstrap first tenant admin
Tenant administrator	@f2p/identity	Create/edit/disable users; assign roles; link Person records
End user	Planning, Hours, WBS, ...	Book hours, view components, edit activities (permission-gated)

User states

State	Meaning	Typical transition
Invited	Pre-provisioned by tenant admin; not yet signed in	Admin creates user with email + roles
Pending	Awaiting first successful login	Invite sent or JIT record created
Active	Can sign in and use product per roles	First OIDC / local login succeeds
Disabled	Login blocked; data retained	Admin disable or seat reclaim
Retired	Offboarded; audit trail kept	Admin retires after project end

Login account vs Person

Legacy F2P separates **User** (authentication) from **Person** (workforce entity for hours and assignments). V2 preserves this:

- **Identity.User** — login principal, roles, enabled/disabled
- **Resources.Person** — who appears on timesheets and WBS assignments
- Tenant admin flow should support **create user + link Person** in one wizard where hour booking is required

Provisioning modes (hybrid recommended)

Mode	Tenant admin	When
Admin-provisioned	Creates user before first login; assigns roles	Default — most shipyard customers
JIT from IdP	Upgrades roles after auto-created login	IdP-led onboarding; <code>jitProvisioning=true</code>
Local accounts	Creates username/password	On-prem air-gap; <code>allowLocalLogin=true</code>

Example roles for operational users

Persona	Example permissions
Viewer	<code>Planning.Read</code> , <code>Wbs.Read</code>
Planner	<code>Planning.Read/Write</code> , <code>Wbs.Read</code>
Hour booker	<code>Hours.Write</code> + project read scope
Team lead	Above + hour approvals
Tenant admin	<code>Identity.Admin</code>

Tenant profile flags

Flag	Effect
<code>adminCanCreateUsers</code>	Tenant admin can pre-provision users (recommended true)
<code>jitProvisioning</code>	Auto-create user on first IdP login
<code>allowLocalLogin</code>	Local account UI (on-prem)
<code>seatLimit</code>	Block create when license cap reached
<code>defaultRoleOnJit</code>	e.g. <code>Viewer</code> until admin upgrades

4. Module placement

Artefact	Type	Owns
F2P.Identity	Backend bounded-context module	Auth API, users, roles, permissions
@f2p/identity	Frontend context module	User/role admin screens under Settings
f2p-shell	Host app (not a domain module)	Login, callback, shell chrome, lazy routes
Admin backoffice	Separate app	Tenant provisioning, SSO binding, first admin bootstrap

Canonical source: docs/monolith-modularization/platform-authentication-standard.md · Diagrams: docs/monolith-modularization/diagrams/